

ENSC 254 Lab Assignment 2

Important Logistics:

- Some general grading logistics and lab computer access instructions have been posted on our course website.
- Lab 2 weighs 9% of the final marks. It includes 140 points in total, which will be scaled to 9% of the final marks.
- Lab 2 will be done and graded per 2-student group, as detailed in the lab logistics. Some points breakdown and guideline on workload division are provided later in the description.
- Please make sure your code can compile and run correctly on the lab computers (i.e., FASRLA Linux computers). If your code cannot compile on the lab computers, then you get 0 mark for lab 2. If your code can compile, but cannot run correctly, then you only get the points where your code runs correctly on the lab computers.

Introduction of Lab 2:

The purpose of this assignment is to enhance your understanding of the RISC-V instruction set design, which provides the abstraction/interface between the hardware and software. Specifically, you will develop a disassembler in lab 2, which will decode a binary code (i.e., 0s and 1s) into a human-readable assembly program (i.e., RISC-V instructions). It's challenging and fun.

Instructions That You Need to Support

The instructions that your disassembler must handle are listed below.

Instruction	Type	Opcode	Funct3	Funct7/IMM	Operation
add rd, rs1, rs2	R	0x33	0x0	0x00	$R[rd] \leftarrow R[rs1] + R[rs2]$
mul rd, rs1, rs2			0x0	0x01	$R[rd] \leftarrow (R[rs1] * R[rs2])[31:0]$
sub rd, rs1, rs2			0x0	0x20	$R[rd] \leftarrow R[rs1] - R[rs2]$
sll rd, rs1, rs2			0x01	0x00	$R[rd] \leftarrow R[rs1] \ll R[rs2]$
mulh rd, rs1, rs2			0x1	0x01	$R[rd] \leftarrow (R[rs1] * R[rs2])[63:32]$
slt rd, rs1, rs2			0x2	0x00	$R[rd] \leftarrow (R[rs1] < R[rs2]) ? 1 : 0$
xor rd, rs1, rs2			0x4	0x00	$R[rd] \leftarrow R[rs1] \wedge R[rs2]$
div rd, rs1, rs2			0x4	0x01	$R[rd] \leftarrow R[rs1] / R[rs2]$
srl rd, rs1, rs2			0x5	0x00	$R[rd] \leftarrow R[rs1] \gg R[rs2]$
sra rd, rs1, rs2			0x5	0x20	$R[rd] \leftarrow R[rs1] \ggg R[rs2]$
or rd, rs1, rs2			0x6	0x00	$R[rd] \leftarrow R[rs1] \vee R[rs2]$
rem rd, rs1, rs2			0x6	0x01	$R[rd] \leftarrow (R[rs1] \% R[rs2])$
and rd, rs1, rs2			0x7	0x00	$R[rd] \leftarrow R[rs1] \& R[rs2]$

lb rd, offset(rs1)	I	0x03	0x0		$R[rd] \leftarrow \text{SignExt}(\text{Mem}(R[rs1] + \text{offset}, \text{byte}))$
lh rd, offset(rs1)			0x1		$R[rd] \leftarrow \text{SignExt}(\text{Mem}(R[rs1] + \text{offset}, \text{half}))$
lw rd, offset(rs1)			0x2		$R[rd] \leftarrow \text{Mem}(R[rs1] + \text{offset}, \text{word})$
addi rd, rs1, imm		0x13	0x0		$R[rd] \leftarrow R[rs1] + \text{imm}$
slli rd, rs1, imm			0x1	0x00	$R[rd] \leftarrow R[rs1] \ll \text{imm}$
slti rd, rs1, imm			0x2		$R[rd] \leftarrow (R[rs1] < \text{imm}) ? 1 : 0$
xori rd, rs1, imm			0x4		$R[rd] \leftarrow R[rs1] \wedge \text{imm}$
srli rd, rs1, imm			0x5	0x00	$R[rd] \leftarrow R[rs1] \gg \text{imm}$
srai rd, rs1, imm			0x5	0x20	$R[rd] \leftarrow R[rs1] \gg \text{imm}$
ori rd, rs1, imm			0x6		$R[rd] \leftarrow R[rs1] \vee \text{imm}$
andi rd, rs1, imm			0x7		$R[rd] \leftarrow R[rs1] \wedge \text{imm}$
ecall		0x73	0x0	0x000	(Transfers control to operating system) a0 = 1 is print value of a1 as an integer. a0 = 10 is exit or end of code indicator.
sb rs2, offset(rs1)	S	0x23	0x0		$\text{Mem}(R[rs1] + \text{offset}) \leftarrow R[rs2][7:0]$
sh rs2, offset(rs1)			0x1		$\text{Mem}(R[rs1] + \text{offset}) \leftarrow R[rs2][15:0]$
sw rs2, offset(rs1)			0x2		$\text{Mem}(R[rs1] + \text{offset}) \leftarrow R[rs2]$
beq rs1, rs2, offset	SB	0x63	0x0		if($R[rs1] == R[rs2]$) $PC \leftarrow PC + \{\text{offset}, 1b'0\}$
bne rs1, rs2, offset			0x1		if($R[rs1] != R[rs2]$) $PC \leftarrow PC + \{\text{offset}, 1b'0\}$
lui rd, offset	U	0x37			$R[rd] \leftarrow \{\text{offset}, 12'b0\}$
jal rd, imm	UJ	0x6f			$R[rd] \leftarrow PC + 4$ $PC \leftarrow PC + \{\text{imm}, 1b'0\}$

All the information is copied from the RISC-V green card for your convenience. You may use the RISC-V green card and instruction specification as a reference. In addition, we also include a “DisassembleFlowchart.pdf” to guide you about the decoding process.

Just like the regular RISC-V architecture, the RISC-V system you're implementing is little-endian. This means, for a given value comprised of multiple bytes, the least-significant byte is stored at the lowest address.

Framework Code

To start the assignment, download lab2.zip from the course website and put it into a Linux machine.

1. `unzip lab2.zip` //decompress the zip file
2. `cd lab2` //go to the lab2 folder
3. `ls` //you can see all the files, including DisassembleFlowchart.pdf

To make your life easier, we have provided the framework code to load a binary program from the input file into the main memory, fetch each 32-bit instruction in binary code, and call the “`decode_instruction(uint32_t instruction_bits)`” function to disassemble the instruction bits. This allows you to focus on parsing the binary code into the instructions and printing them out. Moreover, we have also provided example code to walk you through the process of how to disassemble a few R-type instructions such as `add`, `sub`, and `mul`. A “DisassembleFlowchart.pdf” is also included to guide you about the decode (i.e., disassemble) process.

While working on lab 2, you should thoroughly consult (but not change) the following files:

- `types.h`: C header file for the data types you will be dealing with. It is critical that you read and understand the definitions in `types.h` before starting lab 2.
- `utils.h`: File that contains the format for instructions to print.
- `code/input`: Various input files (i.e., binary programs) to run tests.
- `code/ref`: Golden reference outputs (.solution files) to validate (grade) your disassembler
- `test_disasm.sh`: script to test (grade) your disassembler by comparing your generated outputs (in `code/out`) against the golden reference outputs

You should not need to look at the following files in lab 2:

- `test_utils.c`: this is the unit test code for the functions you are implementing in `utils.c`
- `CUnit-code`: CUnit test framework code
- `install-cunit.sh`: script to install CUnit test
- `riscv.h`: C header file for the functions you are implementing in lab 2 and lab 3.
- `riscv.c`: C source file for the program loader and main function.
- `emulator.c`, `test_emulator.sh`, `emulator_tester.py`: These are for lab 3, which is built on top of lab 2. Details will be provided in lab 3 description.
- `Makefile`: makefile to build and clean up the project

Your TODO list:

In lab 2, you will only need to work on two files: `utils.c` and `disasm.c`. Please do not change any other files. To implement the disassembler in lab 2, you will implement the following functions. The function prototypes have been given; don't change them. Please change the

function bodies, which are indicated by the comment “/* YOUR CODE HERE */”. If you want to add extra helper functions, it’s ok; but don’t change existing function prototypes.

- **Instruction `parse_instruction(uint32_t instruction_bits)` in `utils.c`:** It uses the given instruction (encoded as a 32-bit unsigned integer) and returns the corresponding Instruction (union of structs data type) object. You will have to determine the proper format of the given instruction, choose the corresponding struct type inside the union, and set the corresponding member of the chosen struct. You will find the RISC-V green card and the example code for R-type instructions particularly helpful here.
- **`sign_extend_number(field, n)` in `utils.c`:** This function interprets the number in field as an n-bit number and sign-extends it to a 32-bit integer. This is a helper function that will be called in other functions in lab 2 and lab 3. You need to pass the unit test.
- **`get__offset(instruction)` in `utils.c`:** For the corresponding instruction type (either branch, jump, or store), this function unpacks the immediate value and returns the number of bytes to offset by. In the cases of branches and jumps, these functions should return the number of bytes to add to the PC to get to the desired label. In the case of stores, the corresponding function will return the offset on the destination address. These are helper functions that will be called in other functions in lab 2 and lab 3. You need to pass the unit test.
- **`print_*(instruction)` in `disasm.c`:** Print the instruction to standard output. You should use the constants defined in the file `utils.h` to properly format your instructions. Failure to do so will cause issues with our autograde script. You should also refer to registers by their numbers and not their names. Example code of `print_rtype` is given for your reference.
- **`write_*(instruction)` in `disasm.c`:** These functions call the `print_*(instruction)` to print the instruction to standard output, depending on different cases. Example code of `write_rtype` is given for your reference; note the given `write_rtype` code is incomplete and only covers add, sub, and mul instructions.

Basic Commands and Testing:

- To build your disassembler, type the following command on a lab computer (all commands are typed when you are inside the lab2 folder):

```
make riscv
```

The built executable is called “riscv”. Note: typing “make” does the same thing.

- To run your disassembler with a test input, type the following command:

```
./riscv -d code/input/R/add.input
```

-d option tells the riscv executable to do the decoding (i.e., disassembling). The input is the “code/input/R/add.input” file. If you use vim to open this file (*vim code/input/R/add.input*), you can see its content “00000433”, which is the hex representation of the instruction. The above command will disassemble this hex representation, and print out the actual instruction onto the screen: “00001000: add x8, x0, x0”. The part before the “:” is the address of the instruction (program counter), which is taken care of by the provided framework code; you can safely ignore it in lab 2. The part after the “:” is the actual instruction after the decoding.

To redirect this output into an output file (“code/out/R/add.solution”), instead of printing it onto the screen, type the following command:

```
./riscv -d code/input/R/add.input > code/out/R/add.solution
```

- To validate the correctness of the above test, type the following command:

```
diff code/out/R/add.solution code/ref/R/add.solution
```

For every input file in Test 2 and Test 3, we have also provided the golden reference file to compare against. You can always use vim to open a golden reference file to check what the solution should look like; based on that, you can fix your own code. If the above command silently ends, congratulations, you got it right; otherwise, it will show you the difference between your decoded instruction(s) and the golden reference. For Test 4, the golden reference file is not provided and you will have to manually decode the instructions to verify your output. You can use gdb or cgdb to help the debugging and code fix. Note we already provided the code to decode add, sub, and mul.

More Testing and Grading:

- **[10 points] Test 1: Unit test.** To perform (partial) unit test of the aforementioned helper functions and the parse_instruction function in utils.c, type the following command:

```
./install-cunit.sh // install the CUnit test framework; this only needs to be done once
```

```
// unless you type “make deepclean”
```

```
touch test_utils.c; make test-utils // build and perform the unit test
```

There are 10 unit tests (starting with “Test: test...”) in total: passing 1 unit test (the above command will print out an “FAILED” or “passed”) gives you 1 point for lab 2. Below is an example print out on the screen.

```
Suite: Testing util functions
```

```
Test: test_sign_extend_number ...passed
```

```
Test: test_parse_instruction_rtype ...passed
```

```
Test: test_parse_instruction_itype ...passed
```

```
Test: test_parse_instruction_stype ...passed
```

```
Test: test_parse_instruction_sbtype ...FAILED
```

```
Test: test_parse_instruction_ujtype ...passed
```

```
Test: test_parse_instruction_utope ...passed
```

```
Test: test_get_branch_offset ...FAILED
```

```
Test: test_get_jump_offset ...passed
```

```
Test: test_get_store_offset ...passed
```

- **[65 points] Test 2: test for each type of instructions**, including
 - [7 points] R-type instructions in code/input/R/R.input. 10 instructions in total in this input file and we have provided the code to decode 3 of them. For the remaining 7 instructions, passing each instruction gives you 1 point for lab 2. The golden reference for this input is in code/ref/R/R.solution. To perform this test, type the following commands


```
./riscv -d ./code/input/R/R.input > ./code/out/R/R.solution
diff ./code/out/R/R.solution ./code/ref/R/R.solution
```

For the following types, the golden reference files are also in the code/ref folder, following the same naming convention and the commands are also similar. A complete set of commands are provided in test_disasm.sh.
 - [19 points] I-type instructions in code/input/I/I.input (I-type except load instructions) and code/input/I/L.input (load instructions). 19 instructions in total (6 load instructions and 13 non-load instructions) and each counts for 1 point.
 - [10 points] S-type instructions in code/input/S/S.input. 10 instructions in total and each counts for 1 point.
 - [10 points] SB-type instructions in code/input/SB/SB.input. 10 instructions in total and each counts for 1 point.
 - [10 points] U-type instructions in code/input/U/U.input. 10 instructions in total and each counts for 1 point.
 - [9 points] UJ-type instructions in code/input/UJ/UJ.input. 9 instructions in total and each counts for 1 point.
- **[45 points] Test 3: test for entire program**, including
 - [24 points] multiply testcase code/input/multiply.input. 24 instructions in total and each counts for 1 point.
 - [21 points] random testcase code/input/random.input. 21 instructions in total and each counts for 1 point.
 - The above two testcases are for end-to-end program tests, where each program includes a mix of different types of instructions. Once you passed the test for each type of instruction, this part of the test should be relatively straightforward.
 - The only new instruction that you need to support is the “*ecall*” instruction.
 - You may need to debug the code to pass these two testcases.
- **[20 points] Test 4: test for blind set of instructions without references**, including
 - [10 points] blind instruction set 1 code/input/blind_set1.input. 10 instructions in total and each counts for 1 point.

- [10 points] blind instruction set 2 code/input/blind_set2.input. 10 instructions in total and each counts for 1 point.
- The above two testcases contain two random sets of instruction, where each set includes a mix of different types of instructions. Some of these instructions cover corner cases of your disassembler and you are not given reference solutions for these two cases. **Hint:** They contain at least one invalid instruction.
- If you had thoroughly followed the decoding process for all instructions listed in the aforementioned table, this part of the test should be straightforward.
- The golden reference file is not provided (that's why it's called a blind set) and you need to manually decode the instructions to verify the correctness of your output.

Assignment Submission:

Your lab assignment 2 will be submitted electronically. You will need to submit a single ***YOUR_LAB_GROUP.zip*** file (e.g., LA01_01.zip). This format makes it easy for the TAs to do auto-grading. **If you fail to comply with this format, you will lose 10% of the total points of this lab.** To zip your files in Linux,

1. *Go to your lab2 folder*
2. *make deepclean //make sure you clean up your files*
3. *cd .. //go one level up*
4. *mv lab2 YOUR_LAB_GROUP //rename your lab2 folder using your lab group (e.g., LA01_01)*
5. *zip -r YOUR_LAB_GROUP.zip YOUR_LAB_GROUP //zip all your lab2 files into a single .zip*

Submission:

You need to meet the deadline: every 10 minutes late for submission, you lose 10% of the points; that is, 100 minutes late, you will get zero for this lab.

Lab Demonstration:

To finalize your lab 2 points, you will have to demo your lab code to your TA in the lab sessions that you enrolled. Due to the large enrollment, TA will **randomly** select some students (about 1/2 student groups) to do the lab 2 demo. If you are NOT selected to do the demo, the auto-graded points are your final points.

For those who are selected to do the demo, each student group has around 10 minutes to explain their code to the TA. Only code from your submission is allowed in the lab demo. If you fail to do the demo (without a medical note), you will be awarded zero on this lab assignment. **If it is determined that you do not understand your code being evaluated (the part you are responsible for based on your workload distribution), you will be awarded zero and**

considered as CHEATING on this lab assignment. Also please show up in the demo day on time (TA will send out your scheduled time), otherwise you will lose 10% of the lab 2 points.